Reclaiming Closed Service Robots: A Hypothesis-Driven Methodology for Non-Destructive Protocol Reconstruction

***Abstract*—Commercial service robots are increasingly sold as closed Android–ROS systems: no public SDK, no message schemas, and a vendor cloud that the owner cannot replace. A laboratory that owns such a robot cannot modify its interaction behaviour, nor establish which commands the machine accepts. We present a non-destructive methodology for recovering that command surface without vendor cooperation. Structured as four falsifiable hypotheses regarding protocol observability and verification, our approach is evaluated through a seven-phase pipeline that triangulates network mapping, `rosapi` introspection, static analysis of the vendor application, and passive broker observation. We validate the methodology on a commercial reception robot, recovering enough of its command surface to rebuild teleoperation, autonomous navigation, speech output, and visitor interaction as an edge platform operating without the vendor cloud. Our primary empirical finding concerns verification rather than discovery: a transport-level acknowledgment reveals nothing about physical execution. In 17 logged field commands, the execution outcome was dictated entirely by the reported localization state. All ten failures occurred while unlocalized, and all seven successes occurred while navigating. Addressing scheme choice showed no effect, and a controlled bench comparison confirms that two addressing schemes expected to differ succeed equally often ($p = 1.00$) in equal time ($p = 0.76$) once preconditions hold. Reaching this conclusion required correcting our own instrumentation, which initially invoked an unexposed firmware service. We report field indicators with confidence intervals, a capability comparison against the vendor application, and transferability conditions for other closed platforms.**

***Index Terms*—Service robotics, reverse engineering, protocol reconstruction, vendor lock-in, ROSBRIDGE, MQTT, embedded Android, edge computing, human-robot interaction**

I. Introduction

A laboratory that buys a commercial service robot buys a sealed appliance. The vendor ships an Android application, a cloud account, and a support contract; it does not ship the message schemas that would let the owner send the robot anywhere the application does not already go. For research groups this is a hard limit: the platform in Fig. 1 navigates, speaks and greets visitors perfectly well, but only in the ways its manufacturer anticipated.

Why this is hard ? Three obstacles compound, and none of them is peculiar to this vendor. First, the robot is not one computer but two: an Android head and a ROS chassis, joined by an undocumented internal link. No single point of observation covers the whole command surface, and each one is blind in a different place (Section IV). Second, there is no white-box route. Shells on the chassis are closed, and repeated authentication attempts lock the account, so we cannot simply read the running system. Third, and least obvious, the software layer that accepts a message is not the layer that turns the wheels. A command can be accepted, acknowledged and then quietly dropped. Treat the acknowledgment as proof and the result is a protocol description that is confidently wrong. This is the reason naive fuzzing of the exposed interfaces gets nowhere on such a platform.



Fig. 1. The study platform running our reconstructed visitor interface on its own touchscreen. The head is an Android computer; the mobile base underneath is a separate Linux/ROS machine. Neither exposes a documented interface to its owner. Identifying marks are blurred.

Research question. *Can a non-destructive methodology recover the command surface of a closed Android–ROS robot without documentation, and what evidence is required before a recovered command may be considered validated?*

We formulate four hypotheses, stated here and resolved in Section VI:

- **H1 (application-as-specification).** Static analysis of the vendor's own Android application yields the wire protocol the chassis actually speaks.
- **H2 (broker-as-command-channel).** The exposed MQTT broker can issue motion commands, as the WebSocket bridge does.
- **H3 (map-artifact reuse).** Reusing the vendor's own map annotations is more reliable than re-deriving goal coordinates offline.

- **H4 (transport ≠ execution).** A transport acknowledgment does not imply that the chassis executed the command.

Contributions.

C1. A reproducible seven-phase reverse-engineering methodology for closed Android–ROS robots, organised around falsifiable hypotheses and a triangulation rule, together with the effort it required in practice (Sections IV and VI).

C2. A recovered command surface for a commercial reception platform and an edge-first architecture built on it, including a speech channel obtained through Android inter-process communication after every protocol-level channel had been eliminated (Section V).

C3. Field evidence that execution must be verified against state telemetry rather than transport acknowledgments, with the resulting design rule, quantified indicators, and an analysis of which findings should transfer to other closed platforms (Sections VI and VII).

All work reported here was carried out on hardware owned by our institution, for interoperability purposes, without redistributing vendor code or circumventing any licensing mechanism; Section VII details the ethical and legal framing.

II. Related Work

A. Re-engineering commercial robot platforms

The closest precedent is Kim *et al.* [1], who re-engineered the software architecture of a commercial home service robot. Their study follows much the same arc as ours, but one difference decides everything: they had the manufacturer's source code and its engineering team, while we start from a sealed product. Mühe *et al.* [2] recovered a proprietary industrial robot language, again working from artifacts the vendor had published. Both begin with more material than we do, and the methodology below exists mainly to make up for that shortfall.

B. Vendor lock-in and interoperability

The trade-off between proprietary and open platform strategies is long established [3], and service robotics reproduces it one product at a time. Glauser *et al.* [4] report healthcare deployments held back by outdated and restrictive vendor SDKs, and propose an interoperable programming interface as the remedy. Middleware bridges such as ROS–YARP [5] address the adjacent problem of connecting two interfaces that are both *known*. In every case the interface itself is taken for granted. The question we take up comes earlier: how does a laboratory obtain one at all, on a robot it has already bought?

C. Black-box access to ROS systems

ROSBRIDGE was designed to let non-ROS clients reach a ROS graph over WebSocket [6], [7], normally deployed by the integrator who controls that graph. DeMarinis *et al.* [8] inverted this perspective, scanning the Internet for exposed ROS systems and showing that sensors and actuators can be reached without any knowledge of the node graph; Dieber *et al.* [9] treat the same openness as an application-level security problem. We occupy a similar position with the sign reversed: we too arrive at a vendor-deployed bridge as an unknown client, but constructively, and on hardware we own.

TABLE I
ROUTES TO A CONTROL INTERFACE ON A CLOSED SERVICE ROBOT, AND WHAT EACH PRESUPPOSES

Route	Offl.	Insp.	N.-d.	Presupposes
Vendor SDK [4]	varies	no	yes	A published SDK, kept current by the vendor
Vendor cloud API	no	no	yes	A subscription and a network path off site
Bridge integration [5], [6]	yes	yes	yes	Control of the ROS graph, and known message definitions
Re-architecting with vendor support [1]	yes	yes	yes	Source code and access to the vendor's engineers
Companion-app recovery [2], [13]	—	yes	yes	An artifact that already speaks the protocol
Firmware replacement	yes	yes	**no**	Root access; voids warranty, unrecoverable if it fails
This work	**yes**	**yes**	**yes**	**Only lawful possession of the robot**

D. Protocol recovery from companion software

Extracting protocol specifications from observed traffic is a mature security technique [10], and Android analysis has focused on information flow within an application [11] or between applications [12]. The practice of reading a device's companion app to recover its wire protocol is established in the IoT community [13]. Hypothesis H1 imports that practice into robotics, where the companion application controls a second, physically separate computer.

E. Edge robotics and on-device interaction

Cloud robotics assumes a reliable path to remote compute [14]. We had the opposite constraint. The robot's own network guarantees no Internet route, so speech and recognition had to run on the head, which pushed us towards closed-vocabulary recognition [15], [16] and on-device face embeddings [17]. Language-model agents [18], [19] assume an open action API is already available. On a closed robot that API has to be recovered first, which is the work reported here.

Table I sets out the gap from the integrator's side, by asking what each route to a control interface requires the vendor to have provided first. Every neighbouring approach either starts once the interface is known, or buys openness at a price a single owned unit cannot pay; what we describe is the step before all of them.

III. Platform and Access Model

The study platform is a mass-produced indoor reception robot, sold as a turnkey product with a companion Android application and a subscription cloud service. It pairs a Linux/ROS chassis [20] carrying SLAM, a planner, LiDAR and RGB-D sensing with an Android 7.1 head that drives a 15.6 inch

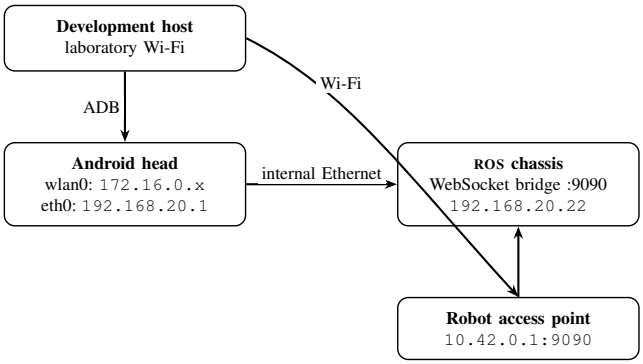


Fig. 2. Three network segments on a product marketed as a single device. Conflating the head's DHCP address with the chassis address was the most frequent operator error we observed.

touchscreen, cameras and speakers. We refer to it generically throughout, since nothing in the methodology depends on the particular model and we have no interest in singling out one manufacturer. The two computers run independent network stacks, a fact no vendor document states and no application string reveals.

Network mapping turned up three segments (Fig. 2), and that was already a result in itself. Two channels are reachable from outside the vendor stack: a WebSocket bridge on port 9090 and an MQTT broker on port 1883. Shell access to the chassis is closed. We observed that repeated authentication attempts trigger an account lockout, and went no further down that road, partly because it would break the non-destructive principle stated below and partly because we had one unit and bricking it would have ended the study. Two vendor packages sit on the head, a welcome application and a deployment tool, and both link against the same chassis-communication library. We treat that library as the *de facto* protocol specification. It is the empirical basis for H1.

IV. Methodology

A. Principles

Four rules govern the procedure.

- **P1: observe before publishing.** We listen passively before injecting anything, because there is no way to predict how an undocumented safety-critical system will react.
- **P2: verify effects, not acknowledgments.** A command counts as validated only once an independent state topic changes. This is the working form of H4.
- **P3: triangulate.** We accept a finding only when two independent sources agree, drawn from network traces, introspection, decompiled application code and field logs.
- **P4: remain non-destructive.** No firmware modification, no root, no persistent change to vendor partitions. This is an ethical commitment, and with a single unit it is also what makes the work repeatable.

B. Vantage points and their blind spots

The whole methodology exists because no single observation point tells us enough (Fig. 3). Introspection lists names, but says nothing about preconditions. Traffic capture shows what the vendor application happened to send during one session, which is not the same as what the chassis would accept. Decompiled code gives intent and the implicit setup steps, but leaves open whether a given path is reachable on this firmware. Triangulating between the three is not a stylistic preference; it is the only way to cover what any one of them misses.

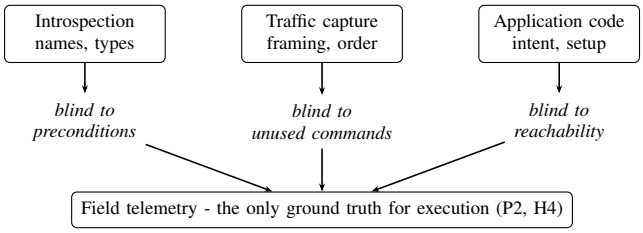


Fig. 3. Each vantage point answers a different question and is blind to the others. A recovered command is accepted only after state telemetry confirms a physical effect.

TABLE II
Reverse-engineering pipeline

#	Phase	Evidence produced	Hyp.	Stopping criterion
1	Network mapping	Segment map, open ports	-	All hosts and reachable ports enumerated
2	Introspection	Topic and service inventory	H1	Inventory stable under a fixed set of running subsystems
3	Passive broker	Payload classes on the broker	H2	No command payload after repeated sessions
4	Application audit	Names, types, implicit setup steps	H1	Every observed name located in code
5	Frame capture	Message framing and ordering	H1	Client reproduces vendor framing
6	Edge integration	On-device services and bridges	-	Capability reachable without a host PC
7	Field validation	State transitions, pose deltas	H3, H4	Effect confirmed on independent topic

C. The seven-phase pipeline

Table II gives, for each phase, the evidence it produces, the hypothesis it bears on, and the condition under which we stopped. The stopping criteria are not a formality. With several hundred topics on the wire, an exploration without them simply does not converge.

We kept the negative results instead of discarding them: a legacy velocity topic with no subscriber, two candidate speech topics, three HTTP ports on the head, and the broker command path of H2. Each one closed a branch of the search, which is worth as much as an open one.

We report the cost of the method because a procedure nobody can budget for is of little use to anyone else. Phases 1–6 took two weeks and phase 7 a further week, spread over eight sessions with the physical robot and carried out by one full-time engineer. We decompiled two applications, and archived full introspections rather than counts, which turns the phase 2 stopping criterion into something testable.

Within a session the inventory is exactly reproducible: two introspections 45 s apart returned set-identical results, 448 topics, 334 services and 64 nodes. Across sessions it is not, but it varies in a structured way. An inventory taken three weeks earlier is a strict subset of the later one: nothing disappeared, and the 44 topics and 13 services that appeared belong without exception to one subsystem, the downward-facing depth camera, which was not running on the earlier date. The upward-facing camera's 115 topics were identical on both dates. The phase 2 criterion should therefore be read as stability under a fixed set of running subsystems, not as a single number; an integrator who counts once and assumes the graph is closed will miss whatever was idle that day.

The exploration scripts, the recovered interface description and the logs behind every measured result reported here are released with this paper [21]; the two indicators in Table III that carry no archived log are marked as such rather than implying an archive that does not exist.

V. Recovered Command Surface and Edge Platform

A. Command and telemetry

Motion is commanded by advertising a multiplexer input and then publishing twist messages on it at roughly 10 Hz. The vendor application quietly sets a manual-control mode when a session opens. Our early builds did not, and every command we sent was dropped without comment. That precondition was visible in the decompiled application and in no traffic capture we took, and it was our first concrete encounter with H4. Autonomous motion comes in two forms: a coordinate goal, and a named-annotation service backed by the vendor's marker manager. One status field reports the state machine gating both, with values for unlocalized, ready, navigating, arrived and error. We throttle subscriptions per topic to keep the bridge usable over the robot's Wi-Fi.

One detail cost us time and is worth passing on. A single vendor message type, carrying one integer field, is reused across several unrelated services, each giving that integer its own private meaning. Call one of them with empty or guessed arguments and it returns success, then either does nothing or does the opposite of what was intended. We recovered the correct arguments through the same introspection route that supports H1. Interfaces reconstructed from the outside are prone to this, and an acknowledgment offers no protection against it.

B. Edge architecture

The reconstructed platform is edge-first by necessity, since the robot's network offers no guaranteed Internet path. A domain layer hides the robot behind one interface with two implementations, simulated and real, which kept ordinary software defects separate from genuine protocol uncertainty. Above it, an application layer exposes REST and WebSocket endpoints and runs either on a development machine or on the Android head, so a visitor session needs no external computer. A presentation layer serves the visitor interface of Fig. 1 and the operator console.

C. Speech through inter-process communication

Speech was the hardest capability to recover, and at the protocol level the search simply failed. No topic accepted text, no such service existed, the head's HTTP ports were closed, and the broker carried telemetry only. The reason turned out to be architectural rather than accidental. The vendor implements speech inside the Android application layer, over opaque IPC to a bundled engine, so it was never a robot-level capability at all and no amount of protocol search would have found it. The answer was to stop looking sideways and move up a layer. A 50 kB companion application registers a broadcast receiver [12] and hands incoming text to the platform's native speech engine:

```
am broadcast -n com.cybel.ttsbridge/.SpeakReceiver
  -a com.cybel.ttsbridge.SPEAK --es text "Welcome."
```

The move itself generalises. Whenever a vendor implements a function in the application layer, no protocol-level search will reach it, and the host operating system is where the capability has to be picked up instead.

D. Interaction layer

Visitor interaction runs entirely on the head. We abandoned free dictation early, because a compact offline model has no pronunciation for the proper nouns that matter on a campus; recognition now works from a closed vocabulary rebuilt at request time from the deployed annotations and the question set. The same limitation caught the wake phrase, built from the platform's invented name and consistently decoded as an unrelated word the model did know, until we replaced it with a phonetically close phrase of real words. Face re-identification runs at roughly 1 Hz and sends embedding vectors only, never images.

VI. Experimental Evaluation

Our samples are small, so we give proportions with Wilson score intervals [22], [23], which stay well behaved at these sizes, and we state every trial count.

A. H1 confirmed, H2 rejected

Every topic and service name we later exercised on the wire had first been found in the decompiled vendor library, and so had preconditions like the manual-mode gate that no traffic capture ever showed us. There is a structural reason for this. The vendor's own application has to speak the real protocol in order to drive the chassis, so its compiled code is a lower bound on the command surface in a way that documentation, which may describe intentions rather than behaviour, is not. We did not test the converse. Whether commands exist that the application never uses remains open, and Section VII records it as a limitation.

H2 is rejected. Wildcard subscription across several sessions returned only one telemetry topic, and no payload we replayed changed the robot's state: the vendor library shows

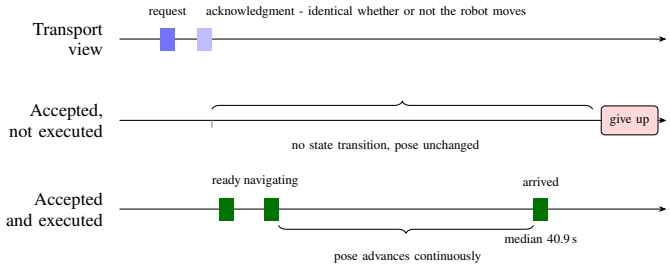


Fig. 4. The transport lane is identical whether or not the command executes. Only an independent state topic distinguishes the two lower cases. Drawn from the field record of Section VI-B, where all ten failures were acknowledged and none moved the robot.

its broker client wired solely to telemetry, the command path running exclusively over the WebSocket bridge. We report the rejection because MQTT is a reasonable first guess for anyone approaching a machine like this, and rediscovering that it leads nowhere is expensive.

B. H4: transport acknowledgment is not execution

The bridge acknowledges any syntactically valid message it can route to a subscriber. That acknowledgment is produced above the chassis safety and mode-arbitration layer, which means it certifies the first hop and nothing beyond it (Fig. 4). Across 17 logged navigation commands, the outcome followed the reported localization state exactly: all ten failures happened while unlocalized, all seven successes while navigating. The same split holds for both goal-addressing schemes: coordinate goals failed five times and succeeded once, named annotations failed five times and succeeded six. The addressing scheme explains nothing here; the precondition state explains everything, and every one of the failed commands had still been acknowledged.

C. H3: addressing scheme versus preparation

Testing H3 took three attempts, and the two failed ones are the more instructive. Our first two campaigns recorded the annotation route failing in every trial against a coordinate route that always succeeded, which inverts H3 and also contradicted daily operation. Introspection dissolved the contradiction: the service our client called does not exist on this firmware, and the fallback we then tried declares one argument, whose name matched none of the three we were sending. We had measured a malformed call to an absent service, not a navigation strategy.

With the call corrected, the comparison is unambiguous (Fig. 5 (a)). Both arms navigated to the same annotated target from a withdrawal point 9.1 m away, automatic mode set and telemetry subscribed. Coordinate goals succeeded 10 times out of 10, median 47.0 s; annotation goals 10 out of 10, median 45.3 s. Neither reliability (Fisher, $p = 1.00$) nor completion time (Mann–Whitney, $p = 0.76$) separates them.

H3 is therefore not supported, and not for the reason the earlier data suggested: annotation reuse is not more reliable, because once the preconditions hold both schemes address the same planner equally well. This strengthens the paper's central claim rather than weakening it. Reuse remains preferable operationally, since it avoids drift between offline coordinates and the live map, but that is a maintenance argument, not a reliability one.

D. System indicators

Table III collects the measured indicators, each with what it is meant to settle; an indicator that settles nothing is padding, and we left several out.

Guided-tour operation is the longest-running of them, and Fig. 6 reports all 47 instrumented sessions over ten weeks rather than the ones that succeeded. Completed tours have a median duration of 10.1 min. Legs within a tour are not independent, so the leg-level interval is the optimistic reading. Two things in that figure are worth saying out loud. The decisive variable is the stability of the map artifacts rather than anything in our stack. And in 309 legs to its right, exactly one failed: a goal issued, acknowledged, and twenty seconds later the robot had not moved. That single failure is the H4 event of Section VI-B, occurring unprompted in service.

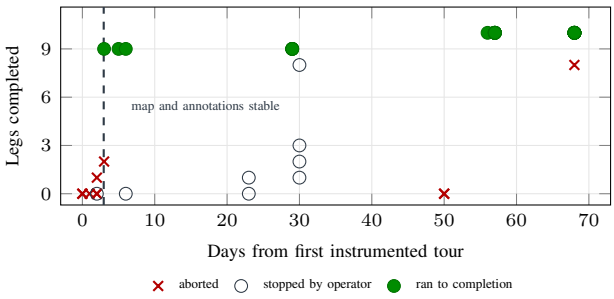


Fig. 6. All 47 instrumented guided tours over ten weeks. To the left of the rule, while the map and its annotations were still being edited, one navigation leg in seven succeeded and no tour finished. To the right, 308 of 309 legs succeeded and 29 tours ran end to end. Operator stops are not failures: every leg they had attempted had succeeded. The single late failure is an H4 event, described in Section VI-B.

The interaction layer contributes three negative results, bounding what the platform delivers. Question matching succeeded in 27 of 48 trials (56.2 %), the failures being near misses rather than outright non-matches, arguing for a wider synonym set over a different threshold. The wake phrase, built from ordinary words so the recogniser would accept it, is ordinary enough to trigger by accident: 20 of 55 logged events opened the microphone with no utterance following, eighteen of them followed by no speech at all, so the cheap remedy is silence rather than a prompt. Finally, the interaction loop is two distributions, not one: 1.3 s median for a local lookup ($n = 9$), 22 s when it also drives the robot ($n = 13$), because preparation runs inside the loop; averaging them, as an earlier revision did, describes neither.

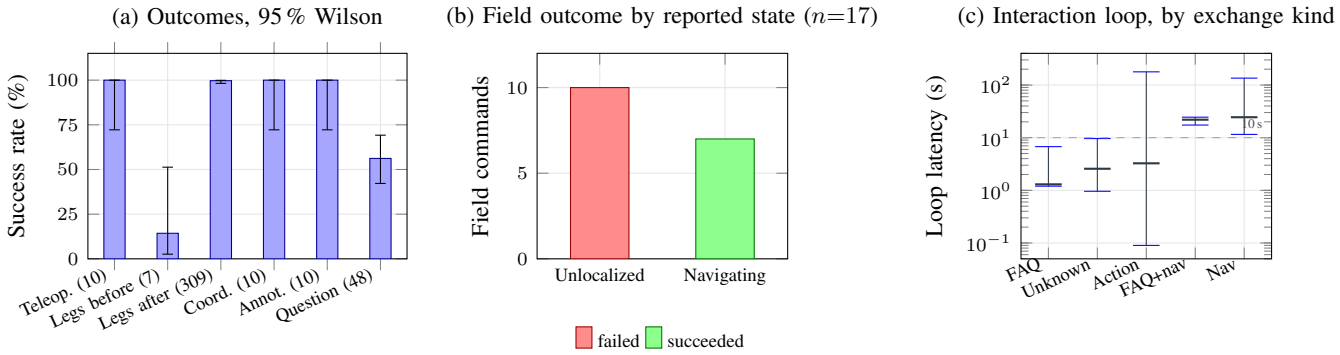


Fig. 5. (a) Every success rate reported in this paper, with its interval. The two leftmost tour bars contrast guided-tour legs before and after the map and its annotations stopped changing, the largest effect we measured; the coordinate and annotation arms of the bench comparison are indistinguishable ($p = 1.00$). (b) In the field record, the state the robot reports separates success from failure perfectly, while the addressing scheme does not separate them at all. (c) Median and range of the interaction loop per exchange kind, log scale. Exchanges that only look up an answer sit below 10 s; those that also move the robot sit above it, because the preparation sequence runs inside the loop. A single pooled latency would describe neither group.

TABLE III

MEASURED INDICATORS, JUNE–JULY 2026, AND WHAT EACH ONE IS FOR. LEGS WITHIN A TOUR ARE NOT INDEPENDENT OBSERVATIONS, SO THE LEG-LEVEL INTERVAL IS OPTIMISTIC. ROWS MARKED † ARE CAMPAIGN READINGS FOR WHICH NO MACHINE-READABLE LOG WAS ARCHIVED.

Indicator	n	**Value**	**95 % interval**	**What it establishes**
Teleoperation success	10	100 %	72.2–100 %	That the manual-mode precondition recovered from the vendor application (H1) is the correct one: without it the same commands were dropped silently
Guided tours run end to end	29	–	–	That the full preparation path works unattended and repeatably over eight weeks, including ten consecutive tours in one campaign
navigation legs, after map stable	309	99.7 %	98.2–99.9 %	The same at leg resolution: not one navigation failure once the map and its annotations settled
navigation legs, before	7	14.3 %	2.6–51.3 %	What the same pipeline did while the map was still changing
Tour duration (median)	29	10.1 min	9.3–13.1 min	The operating envelope of a complete visitor session, for anyone sizing a deployment
Coordinate goal, bench	10	100 %	72.2–100 %	Arm A of the H3 comparison: goals addressed by coordinate; median 47.0 s
Annotation goal, bench	10	100 %	72.2–100 %	Arm B: same target, same conditions; median 45.3 s. Separates from arm A neither in reliability ($p = 1.00$) nor in time ($p = 0.76$)
Repeat-question match	48	56.2 %	42.3–69.3 %	The ceiling of the closed-vocabulary matcher, and the main negative result of the study
Speech-bridge latency (median)	10	702 ms	621–1130 ms	The price of routing speech through the host operating system instead of the vendor engine
Question exchange (median)	9	1.3 s	1.0–6.8 s	The interaction loop when the reply is a local lookup
Navigation exchange (median)	13	22 s	11.6–135 s	The same loop when the reply also drives the robot: the preparation sequence dominates, and the two kinds must not be averaged together
Wake-phrase false triggers	55	36.4 %	24.9–49.6 %	The cost of substituting a real-word phrase for an out-of-vocabulary product name
Local REST latency†	–	< 100 ms	–	That the reconstructed stack adds no delay a visitor would notice on top of the bridge
Wi-Fi round-trip time†	–	–	89–1654 ms	Why telemetry is throttled per topic, and why the round trips removed in Section VII-A were worth removing

E. Comparison with the vendor application

Table IV compares the two systems capability by capability. It is not a controlled user study, and where both provide a function we make no claim about which does it better. The lower block lists the areas where the vendor keeps a real advantage, most of which follow from resources a laboratory does not have.

TABLE IV

CAPABILITY COMPARISON WITH THE VENDOR APPLICATION

Capability	**Vendor**	**This work**
Teleoperation, guided tour	yes	yes
Speech synthesis and recognition	yes (cloud)	yes (on-device)
Face re-id., question answering	yes (cloud)	yes (on-device)
Operates without Internet	no	yes
Inspectable, modifiable, self-hostable	no	yes
Multi-floor navigation, fleet console	yes	no
Updates and support under warranty	yes	no
Recognition across languages	yes	partial

VII. Discussion

A. What the study establishes

The methodological lesson is that discovery and verification are two different problems, and on a closed platform the second is much the harder. Recovering names is fairly mechanical: introspection plus a decompiler produce a large candidate surface in a matter of days. Establishing that any of those commands does something is where the time actually goes, because the interface reports success in exactly the same way whether or not the machine moves. Our field record puts this as starkly as we could have wished. The outcome was settled by a state field, and not at all by the command variant we had spent weeks trying to tell apart.

The same discipline has to be turned on the measuring apparatus, and this is where we were slowest to learn. Two of our campaigns recorded a navigation strategy failing when in fact our client was calling a service the firmware never exposed, and a later verification script watched a status topic whose numbering belongs to the underlying planner rather than to the vendor. In both cases the transport reported nothing wrong. An integrator who trusts a recovered interface without introspecting it will not merely write faulty control code; they will collect faulty evidence about it, and publish it.

From this follows a design rule, and we tested its price rather than assumed it. An ablation removed the preparation sequence entirely and compared ten such trials against ten with it kept, same start, target and 9.1 m route as H3. Both arms succeeded ten of ten, median 41.6 s with preparation, 41.2 s without (Mann–Whitney, $p = 0.50$): from an already-localized state the sequence costs nothing measurable. Its value is insurance against degraded states we could not safely induce on a robot in daily service, not speed, and the ablation leaves that case untested rather than closed.

B. Generalization

H1 should hold wherever the vendor's companion application is the primary control surface, since that application has no choice but to speak the true protocol. H4 should hold on any platform that places a safety or mode-arbitration layer between the message bus and the actuators, which is the norm for commercial mobile bases. The rejection of H2 is specific to this manufacturer, and another may well command over a broker; what carries across is the triangulation procedure that settled the question in a single audit instead of weeks of intermittent failures. Concrete names, gate semantics and naming conventions do not transfer at all, and have to be rediscovered, which is what the pipeline is for. We have run the method on one platform, and cannot yet support that it transfers to a second.

C. Ethics and data protection

The robot belongs to our institution and was studied for interoperability. We analysed the vendor application in order to recover message schemas, not to extract or redistribute proprietary assets. Staying non-destructive kept the warranty and the update path intact, and avoided a firmware fork nobody would maintain.

The legal position deserves stating, because reverse engineering invites the question. We circumvented no technological protection measure: the interfaces are exposed by the robot on its own network and the application package was read as installed, so anticircumvention provisions are not engaged. What we did do, decompiling a vendor application to recover the schemas needed to interoperate with hardware we own, is exempted explicitly in major jurisdictions: the reverse engineering provision of the DMCA [24] and Article 6 of the European software directive [25] both permit it for a lawful possessor, solely for interoperability, where the information is not otherwise available and the result is not used to reproduce the original. Our case meets each condition, and we cite these as the recognised international standard rather than as the law of any particular forum.

Face re-identification processes biometric data, and that needed a separate judgement. Pretrained embedding models differ in the provenance of their training data, and several widely circulated conversions descend from a corpus that was withdrawn over consent concerns. We chose a model whose provenance is documented, obtained through a checksum-verified chain, and we record the trade-off here rather than presenting the choice as settled. Enrollment is never automatic. A staff member initiates it in the presence of the person being enrolled, who is told at that moment what is happening, and individual embeddings can be deleted afterwards from the operator interface. Only embedding vectors leave the head. No image is stored or transmitted.

D. Limitations

The evaluation is uneven: some samples are substantial, notably the 180 guided-tour legs, others thin. Ten trials per arm exclude a large difference between the addressing schemes, and we found none, but not a small one; we claim equivalence is not contradicted, not that it is established. We never instrumented session-level connection reliability, so we describe the reconnection mechanism rather than quote a figure we cannot stand behind. The vendor comparison covers capabilities, not measured task performance. H1 was confirmed in one direction only: observed names do appear in the application, but we did not establish that it exercises every command the firmware accepts, so the recovered surface is a lower bound. The wake-phrase rate is a proportion of logged triggers, not a rate per hour, since our logs span calendar time rather than supervised operation. The face-matching threshold is uncalibrated for several visitors at once, and a few interaction-loop timings were negative through clock skew between the recognition app and the backend; we excluded them. All of it comes from one robot, one map, one site.

VIII. Conclusion

We asked whether the command surface of a closed Android–ROS service robot can be recovered without the manufacturer's help, and what evidence that recovery demands.

The seven-phase pipeline produced a working surface on a commercial reception platform and an edge deployment that replaces the vendor cloud for teleoperation, navigation, speech and visitor interaction. The vendor application supplied protocol names and undocumented preconditions (H1); the exposed broker could not carry commands at all (H2). Most importantly, the gap between transport and execution (H4) governed the exercise: the addressing scheme we expected to matter (H3) explained none of our outcomes, while the precondition state explained all of them. Establishing that required correcting our own measurement twice, our clearest evidence for H4: the instrument is as easily fooled as the integrator. Next steps: a second manufacturer, a measured task comparison in place of our capability table, and a language-model planner grounded in these primitives rather than left to rediscover the interface itself.

References

[1] M. Kim, J. Lee, K. C. Kang, Y. Hong, and S. Bang, "Re-engineering software architecture of home service robots: A case study," in *Proc. 27th International Conference on Software Engineering (ICSE)*, 2005, pp. 505–513.

[2] H. Mühe, A. Angerer, A. Hoffmann, and W. Reif, "On reverse-engineering the KUKA Robot Language," in *1st International Workshop on Domain-Specific Languages and Models for Robotic Systems (DSLRob)*, 2010, arXiv:1009.5004.

[3] J. West, "How open is open enough? Melding proprietary and open source platform strategies," *Research Policy*, vol. 32, no. 7, pp. 1259–1285, 2003.

[4] R. Glauser, J. Holm, M. Bender, and T. Bürkle, "How can social robot use cases in healthcare be pushed with an interoperable programming interface," *BMC Medical Informatics and Decision Making*, vol. 23, no. 1, p. 118, 2023.

[5] M. Aragão, P. Moreno, and A. Bernardino, "Middleware interoperability for robotics: A ROS–YARP framework," *Frontiers in Robotics and AI*, vol. 3, p. 64, 2016.

[6] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "Rosbridge: ROS for non-ROS users," in *Proc. 15th International Symposium on Robotics Research (ISRR)*, 2012.

[7] Open Source Robotics Foundation, "rosbridge_suite," http://wiki.ros.org/rosbridge_suite, ROS Wiki.

[8] N. DeMarinis, S. Tellex, V. P. Kemerlis, G. Konidaris, and R. Fonseca, "Scanning the Internet for ROS: A view of security in robotics research," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8514–8521.

[9] B. Dieber, S. Kacianka, S. Rass, and P. Schartner, "Application-level security for ROS-based applications," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 4477–4482.

[10] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, "Prospex: Protocol specification extraction," in *Proc. IEEE Symposium on Security and Privacy*, 2009, pp. 110–125.

[11] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," *ACM Transactions on Computer Systems*, vol. 32, no. 2, pp. 1–29, 2014.

[12] E. Chin, A. P. Felt, K. Greenwood, and D. Wagner, "Analyzing inter-application communication in Android," in *Proc. 9th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2011, pp. 239–252.

[13] D. Giese and D. Wegemer, "Having fun with IoT: Reverse engineering and hacking of Xiaomi IoT devices," DEF CON 26, 2018.

[14] B. Kehoe, S. Patil, P. Abbeel, and K. Goldberg, "A survey of research on cloud robotics and automation," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 398–409, 2015.

[15] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, arXiv:1804.03209.

[16] Alpha Cephei, "Vosk: Offline open-source speech recognition toolkit," https://alphacephei.com/vosk/.

[17] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.

[18] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. 6th Conference on Robot Learning (CoRL)*, 2022.

[19] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter *et al.*, "PaLM-E: An embodied multimodal language model," in *Proc. 40th International Conference on Machine Learning (ICML)*, 2023.

[20] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, 2009.

[21] Anonymous Authors, "Reverse-engineering scripts, edge platform and field logs," https://anonymous.4open.science/r/reconstruct-system-robot, 2026, anonymised.

[22] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[23] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statistical Science*, vol. 16, no. 2, pp. 101–133, 2001.

[24] U.S. Congress, "Digital millennium copyright act, section 1201(f): Reverse engineering," Pub. L. No. 105-304, 112 Stat. 2860, codified at 17 U.S.C. §1201(f), 1998.

[25] European Parliament and Council of the European Union, "Directive 2009/24/EC on the legal protection of computer programs, article 6: Decompilation," Official Journal of the European Union, L 111/16, 5 May 2009, 2009.